COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S4
DATE PERFORMED:	October 21, 2026	**DATE SUBMITTED:**	October 21, 2026
NAME:	Uchi, Mao Sophie S. Verano, James Daniel M.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 14.1 Algorithm Complexity

1. PROCEDURE OUTPUT

Algorithm 1	Insertion Sort
Algorithm 2	Merge Sort

Table 14-1. Best- and Worst-Case Analysis using Theoretical Tools

	Best Case	Worst Case	Analysis
Algorithm 1	n	n^2	Insertion sorting algorithm, as simple as it gets, inserts the current element from the next element, compares all following elements until all elements are in order from least to greatest. What makes O(n) this algorithm's time complexity for the best case? Firstly, the array should be almost or already sorted as its best case scenario, not having to compare most of these elements and returns with a sorted array. What makes O(n^2) this algorithm's time complexity for the worst case? The array is in reverse chronological order, having to compare all of the elements twice as much as the average case scenario.
Algorithm 2	N logn	N logn	Merge sorting algorithm follows the Divide and Conquer approach, meaning the array will be split into subarrays, reclusively sort the elements from the multiple subarrays, and combine them back as a whole sorted array. What makes O(n logn) this algorithm's time complexity for the best case? For starters, the best case scenario for this algorithm is if the array is almost or already sorted, minimizing the division computation to reduce the amount of subarrays that needs to be fused. What makes O(n logn) this algorithm's time complexity also for the worst case? For the worst case scenario, the array input has a reverse sorting order, having to maximize the total number

			of divisions, increasing its performance rate to merge back the subarrays as one.

Table 14-2. Best- and Worst-Case Analysis using Experimental Tools

	Algorithm 1	Algorithm 2
Best Case	`O(n) (0.06 ms)`	`O(n log n) (11.97 ms)`
Worst Case	`O(n^2) (284.56 ms)`	`O(n log n) (13.44 ms)`
Analysis	With both algorithm, we can see that, in algorithm 1 (insertion sort), it is more efficient in the low space array. Meanwhile, algorithm 2 (merge sort), has an equal amount of time complexity despite the amount of array iterations.	

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO A: Contrast and compare the two searching techniques used in class. The required deliverable for this activity would be the theoretical and empirical analysis.

For the theoretical analysis, the deliverable must be:
- Pseudocode Analysis
- Time and Space Complexities

For the empirical analysis, the deliverable must be the filled-out table and your analysis.

Pseudocode Analysis	In the pseudocode analysis for both linear search and binary search algorithm, **linear search** is the common searching algorithm to sequentially search each of the array and iterations. As for example, if an array has a size of 1000 and its target is 998, the linear searching algorithm will iterate 1 - 998 in order for the target to be reached. As for the **binary searching algorithm**, this algorithm consists of specific conditions, where it halves the whole array, and compares the target value if it is higher or equal than the mid value (which is the value that halves

	the array). If the target is higher, it crosses out the left hand side which are the low numbers;, if it is lower, it crosses out the right hand side or the high numbers. Binary search only works if the array is in sorted form.
Time and Space Complexities	The time complexity of the binary search algorithm is **O(log n).** As it halves the array into 2 parts to create a condition for the target value. On the other hand, the linear algorithm has a time complexity of **O(n)**. This interprets that binary search is faster than linear search. The space complexity for binary search and linear search are just the same as it iterates the same array size in the same searching algorithm process.

Input Size (N)	**Elapsed Time for Linear Search (Worst-Case)**	**Elapsed Time for Binary Search (Worst-Case)**
1,000	`N = 1000` `Linear | Elapsed Time: 2300 nanoseconds`	`N = 1000` `Binary | Elapsed Time: 100 nanoseconds`
10,000	`N = 10000` `Linear | Elapsed Time: 21100 nanoseconds`	`N = 10000` `Binary | Elapsed Time: 200 nanoseconds`
100,000	`N = 100000` `Linear | Elapsed Time: 209500 nanoseconds`	`N = 100000` `Binary | Elapsed Time: 200 nanoseconds`

Analysis	From the console output above, the elapsed time for linear search is sequential to the nth size as the worst case elapse time is twice as slow to the nth size while the binary search is logarithmic to the nth size as the worst case elapse time is divided tenfold. With both being in the worst-case implementation, as you can see, linear search multiplies almost ten times as it increases in each array value. While in binary, it is consistently as low and as close as in each iterative n value difference. Even if they both have the worst-case scenario, binary search still handles it with a more efficient approach as it has a different condition that crosses out a huge portion of the array which lessens the amount of value needed to iterate to reach the target value.

Uchi Conclusion

For my conclusion in this activity, I've learned the depth of Algorithmic complexity, and what should be the most optimal algorithm that requires efficiency, responsiveness, space and time complexity. This module is needed to understand the run time of each algorithm by knowing what hardware a computer has, its data representation, how efficient the compiler is, and knowing the complexity of each algorithm.

Verano Conclusion

In accomplishing this laboratory activity, it has enhanced my analytical interpretation of time and space complexity. With the procedure and supplementary sorting and searching algorithm implementation of time complexity duration, with the comparison between best and worst case scenario, it has furtherly helped me understand the relationship between the algorithm's time and space complexity. With their own unique ways in sorting and searching algorithms, learning how long it takes and how much space it takes is crucial, especially in high data processing. With that, I have further developed a deeper analytical learning in time and space complexity.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Google AI Overview**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Code assistance.**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Recall on the searching and sorting algorithms.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.